UNIVERSIDAD NACIONAL DE LA MATANZA · DIIT · INGENIERÍA INFORMÁTICA
GESTIÓN APLICADA AL DESARROLLO DE SOFTWARE II (3665)

TP6 · Arquitectura del Sistema

RooMeet — asignación y monitoreo de aulas de la UNLaM

Repositorio: https://github.com/gadsii-unlam/gadsii-RooMeet (rama `tp6`)

Entrega: martes 10/11/2026

Integrantes: Bosch, Máximo Augusto · Burnowicz, Alejo · López Ferme, Nahuel Ezequiel · Rodriguez, Mariano Lionel · Rosales, Gustavo Mauricio · Vallejos, Franco Nicolás

Documentos de soporte en el repositorio: `docs/arquitectura/arquitectura-mvp.html` (punto 1), `docs/arquitectura/arquitectura-vision.html` (punto 3), `docs/brief.md` versión 5 (punto 6) y `docs/prompts.md` (punto 7).

1. Arquitectura real del MVP

El MVP es una única aplicación web estática (React + TypeScript + Vite) que Vercel entrega una sola vez; después, todo corre en el navegador del administrativo. No hay servidor de aplicación, base de datos, API, autenticación ni conexión con SIU Guaraní. Se organiza en cuatro capas lógicas dentro de `mvp/src`:

- **Presentación y orquestación:** `App.tsx` es el único punto que conecta capas. Mantiene el reducer de sesión y el estado de interfaz, navega por hash entre las cuatro pantallas (Datos, Propuesta, Monitoreo, Cambio) y la pantalla final, e implementa generar, cambiar, deshacer, finalizar y exportar.
- **Estado de sesión:** `state/reducer.ts` (transiciones atómicas de `SessionState`, la única fuente de verdad, en memoria) y `state/storage.ts` (copia JSON validada en `localStorage`).
- **Dominio, funciones puras:** `allocation.ts` (motor por búsqueda con retroceso), `validation.ts` (capacidad, duplicados, solapamiento), `metrics.ts` (métricas del piloto) y `monitoring.ts` (filtros y ocupación).
- **Datos:** `data/demoDataset.ts`, una constante compilada en el bundle con 65 materias de primer año del DIIT y 6 aulas. Nombres, códigos y horas son reales; horarios, cupos, historial y aulas son simulados.

Según la tabla 8.2 del TP3, el dataset y su visualización están **`simulados / hardcodeados`**; el motor, el monitoreo con filtros y la modificación local están **`construidos`**; Guaraní está **`simulado`** (no hay ni adaptador ni mock: el dataset ocupa su lugar). Los dos diagramas siguientes muestran la vista de despliegue y la vista de código. Dos desvíos respecto de la guía de implementación quedan a la vista en el diagrama: las pantallas importan el dataset directamente, sin pasar por `App`, y la lógica de deshacer vive en `App.tsx` y no en el dominio.

Diagrama 1a · Vista general del MVP (despliegue y datos)

Construido · Simulado o hardcodeado (TP3) · No existe en el código · Dependencia: de quien llama hacia quien es usado · Relación simulada

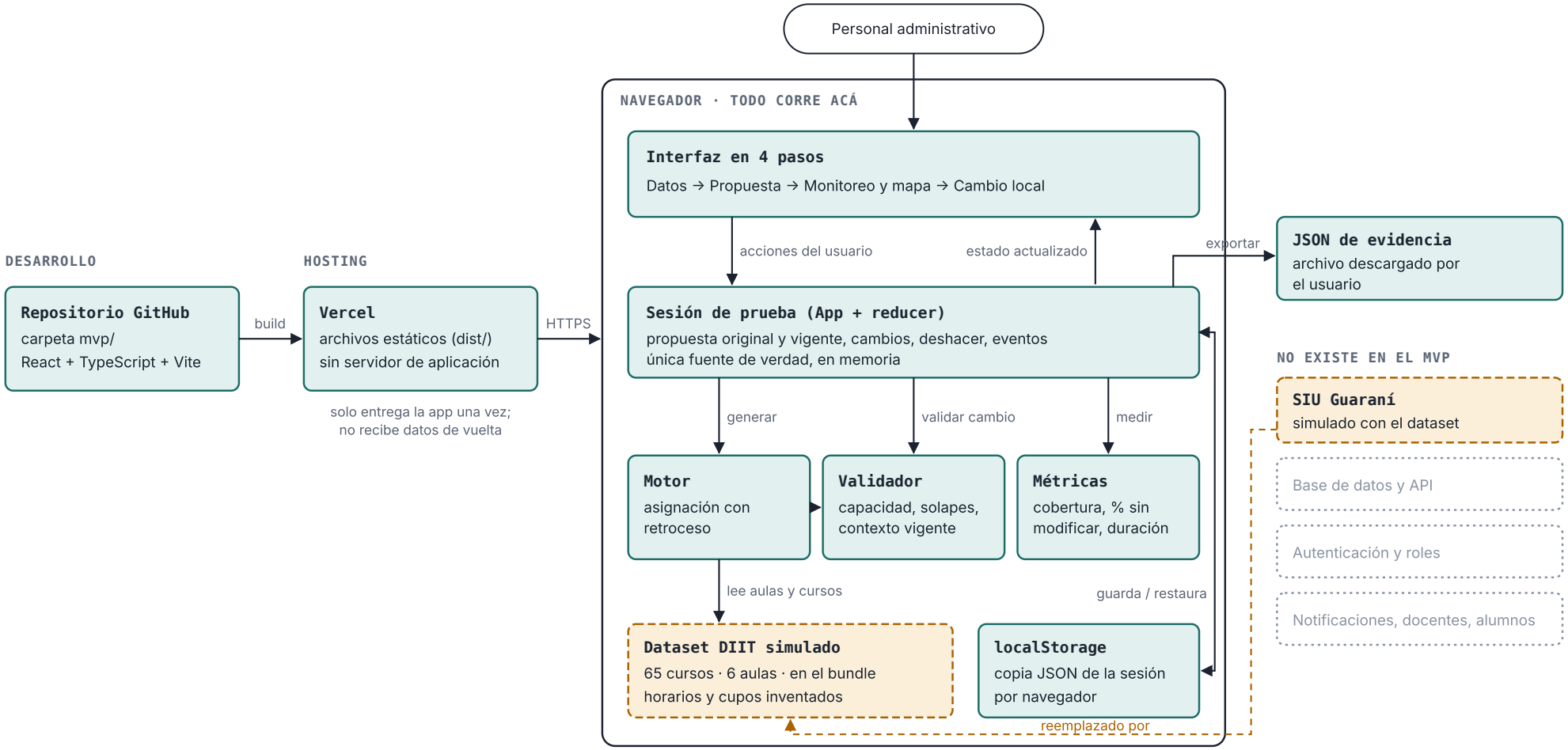
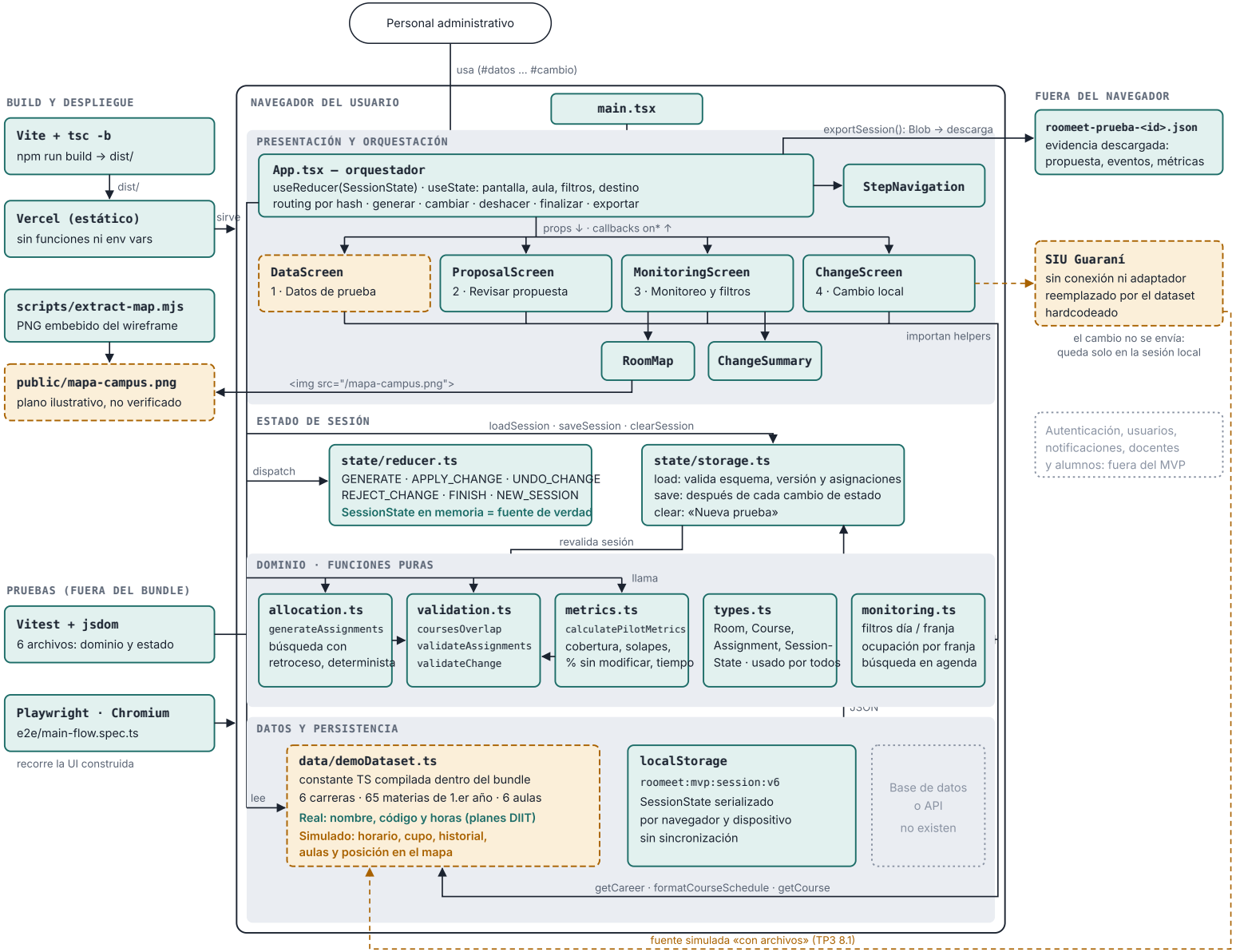


Diagrama 1b · Vista de código del MVP (capas, componentes y dependencias)

Construido · Simulado o hardcodeado (TP3) · No existe en el código · Dependencia · Relación simulada



2. Las decisiones heredadas

La columna «¿Quién la tomó?» se reconstruyó con el historial del repositorio: los commits de `mvp/` y la guía `mvp/MVP-AGENTES.md`, generada con OpenAI Codex el 25/10, que en su línea 7 declara que «las elecciones de stack, algoritmo, dataset ampliado, almacenamiento y medición» son decisiones de implementación de la propia guía, no del equipo ni del relevamiento.

DECISIÓN	¿QUIÉN LA TOMÓ?	¿LA SOSTENDRÍAN HOY?	¿QUÉ DATO DEL TP2 O DEL TP5 LA SOSTIENE O LA CONTRADICE?
La sesión (propuesta, cambios y evidencia) vive sólo en el navegador: estado en memoria más una copia en `localStorage`, sin servidor ni base.	**La herramienta.** Codex la fijó como «Decisión técnica» en `MVP-AGENTES.md` §8. El pedido del equipo sólo decía «dejarlo listo para correr en Vercel».	**Sí para el piloto, no para el producto.** Para una persona y una sesión alcanza; ningún otro actor puede ver ni aprobar lo que se guarda.	**El TP2 la contradice para el producto:** U1 y U3 piden centralizar la información y U2 describe cambios que decide la Secretaría Académica y que afectan a cada departamento: varios actores sobre el mismo dato. Una copia por navegador no se comparte. **TP5:** la prueba con usuarios sigue pendiente, así que no hay dato de uso que la sostenga.
`App.tsx` concentra la orquestación (navegación, deshacer con revalidación, exportar) y las pantallas importan el dataset directamente.	**Nadie, salió así.** La guía pedía un reducer y separar la tabla de propuesta; no menciona dónde vive deshacer ni que las pantallas lean el dataset. No hay commit ni prompt donde se discuta.	**No.** El día que los datos vengan de Guaraní o de una API habrá que tocar cinco pantallas y `App`, porque no hay una interfaz de datos donde enchufar la fuente real.	**Ningún dato del TP2 ni del TP5 tiene que ver con esta decisión.** Se tomó por comodidad de implementación, no por el usuario.
Publicar el MVP como sitio estático en Vercel, fuera de la infraestructura de la UNLaM.	**El equipo.** Pedido explícito en el prompt del 25/10 registrado en `docs/prompts.md`.	**Sí para el MVP, no para la visión.** Sin datos reales, publicarlo afuera es lo más rápido; con datos institucionales, no.	**El TP2 la contradice para la visión:** U1 (Sistemas) dice que ellos proveerán la infraestructura y el mantenimiento, y que Guaraní es la fuente oficial. **TP5:** no conseguimos el aval de las autoridades administrativas y el usuario real no tiene acceso al software; un servicio externo con datos de alumnos tendría todavía menos chances de aprobación.

También los fijó la herramienta y no se discutieron: el stack React + TypeScript + Vite (guía §10), el motor por búsqueda con retroceso (guía §6) y las dependencias declaradas como `latest` en `package.json`, que hacen que cada instalación pueda traer versiones distintas.

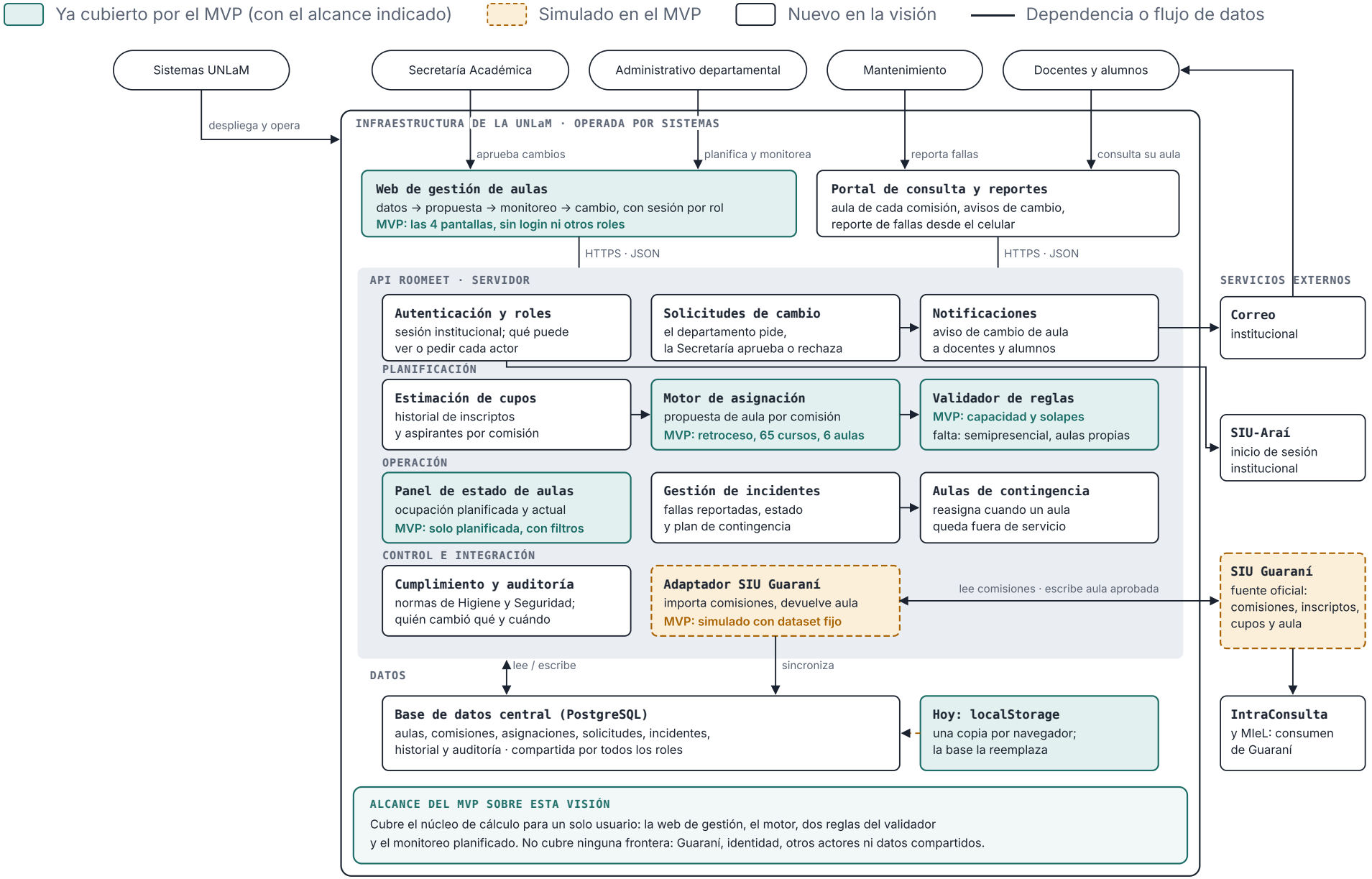
3. La arquitectura de la gran visión

El diagrama 2 muestra el producto completo del TP1, con los actores del TP2: las cinco funcionalidades core (motor, panel de estado, incidentes, contingencia y cumplimiento), las

integraciones previstas y el portal para docentes y alumnos. En verde está lo que ya cubre el MVP y en naranja lo que hoy está simulado.

El salto arquitectónico más grande es pasar de una aplicación que corre sola en un navegador, con datos inventados y un único usuario anónimo, a un sistema dentro de la UNLaM que lee y escribe sobre SIU Guaraní, la fuente oficial («si una comisión no está en Guaraní, legalmente no existe», U1). Ese cambio arrastra a los demás: hace falta un servidor y una base compartida porque varios actores operan sobre el mismo dato, identidad y roles porque la Secretaría Académica aprueba lo que el departamento propone, y un flujo de solicitudes en lugar del «deshacer» local. El MVP está parado en el centro de cálculo de la visión —motor, dos reglas del validador, monitoreo planificado y la web de gestión—, pero no toca ninguna de sus fronteras. Y la prueba del TP5 mostró que el salto es más institucional que técnico: el acceso a esos sistemas no depende del equipo.

Diagrama 2 · Arquitectura de la gran visión, con el alcance del MVP marcado



4. Las tecnologías que agregaría la visión

1. **PostgreSQL como base de datos central.** Reemplaza a `localStorage`: aulas, comisiones, asignaciones, solicitudes, incidentes y auditoría son datos relacionales que varios roles leen y modifican a la vez. Además, la regla central del dominio («una aula no puede tener dos comisiones en la misma franja») se puede reforzar en la propia base con una restricción de exclusión sobre rangos horarios, en lugar de confiar sólo en el código. *Justificación contra el contexto:* según el TP2, Sistemas de la UNLaM (U1) es quien va a proveer la infraestructura y mantener el sistema, y SIU-Guaraní se ejecuta sobre PostgreSQL, por lo que es un motor que ese equipo ya opera. Queda por confirmar con Sistemas la versión y si permiten una instancia propia o un esquema en un servidor existente.
2. **Servicios de integración del SIU: API de SIU-Guaraní y SIU-Araí para la identidad.** El adaptador lee comisiones, inscriptos y cupos de Guaraní y devuelve el aula aprobada; el inicio de sesión y los roles se delegan en el servicio de usuarios del SIU en vez de crear cuentas propias. *Justificación contra el contexto:* Guaraní es la fuente oficial e IntraConsulta y MIeL consumen de él (encuesta de U1), así que cualquier otra fuente divergiría. Y la restricción real que encontró el TP5 —no hubo aval institucional— hace que conectarse por los canales que el SIU ya provee sea lo único que una política de acceso a datos puede aprobar. Mientras ese aval no exista, el adaptador debe aceptar archivos exportados de Guaraní, que es como el TP3 decidió simular la conexión. Qué servicios están habilitados en la UNLaM debe confirmarse con Sistemas.
3. **Servidor en Node.js con TypeScript.** La razón es la familiaridad del equipo, y la declaramos así: es el lenguaje del MVP y el dominio (`allocation.ts`, `validation.ts`, `metrics.ts`) ya está escrito como funciones puras que se pueden mover al servidor sin reescribirlas. La contracara es que Sistemas, que mantendría el producto, trabaja con el ecosistema del SIU; habrá que acordar con ellos si pueden operar un servicio en Node.

5. El componente más riesgoso

El adaptador con SIU Guaraní, es decir, la frontera entre RooMeet y los datos institucionales. Lo elegimos porque es lo que trabó la prueba del TP5: la sesión con usuarios reales sigue pendiente porque no conseguimos el aval de las autoridades administrativas, y el usuario real no tiene ni va a tener acceso al software por los procesos de la Universidad. Si una prueba sin datos reales ya quedó bloqueada, una integración que lee y escribe la fuente oficial depende de permisos que el equipo no controla. A eso se suman tres razones técnicas: hoy no existe ni como interfaz (el dataset está compilado en el bundle y lo leen `App` y las pantallas directamente; ver la decisión 2), por lo que reemplazarlo toca toda la aplicación; un error de escritura cambia la asignación oficial, y U1 advirtió que IntraConsulta puede desfasarse de Guaraní; y los datos reales traen casos que el MVP no modela, como materias semipresenciales que comparten aula alternando días, aulas propias de cada departamento y cupos que cambian por reclamos.

Segundo candidato: el motor de asignación. Con el dataset del MVP (65 cursos, 6 aulas) resuelve en pocos milisegundos y escala bien cuando todo entra: con 520 cursos y 48 aulas tardó unos 50 ms. Pero al leer `allocation.ts` y probarlo encontramos un defecto latente: cuando algún curso no entra en ninguna aula, la rama que «saltea» ese curso no deshace las asignaciones que agregó y el resultado sale corrupto. Agregando 4 comisiones a una franja

saturada (69 cursos), el motor devolvió **236 asignaciones para 69 cursos, con 385 violaciones**, y `App.tsx` lo publica como propuesta porque no lo valida antes, aunque la guía lo exigía. No falló en el MVP porque el dataset se armó para que todo entrara; con datos reales, donde habrá franjas saturadas, fallaría en el primer uso. Es más fácil de corregir que el adaptador, por eso queda segundo.

6. Brief de Producto — versión 5

`docs/brief.md` quedó actualizado a la versión 5 y abre con el párrafo de cambios. Agrega la sección 9 con la arquitectura del MVP, la gran visión, el salto entre ambos, las tecnologías nuevas y las restricciones técnicas del contexto, y actualiza los pendientes. Como la versión 4 no llegó a registrarse en el repositorio, el párrafo de cambios explica también lo que correspondía a esa versión (el MVP construido en el TP5).

7. Sobre el uso de IA en este trabajo

Usamos IA para producir el diagrama de la visión y dar formato a los diagramas del MVP, ordenar la documentación y este informe, explorar alternativas tecnológicas y medir el motor con datos de prueba más grandes. No delegamos la lectura del código para reconstruir la arquitectura real ni la clasificación de las decisiones heredadas: la IA juntó la evidencia del historial (commits, la guía `MVP-AGENTES.md` y `prompts.md`) y el equipo decidió quién tomó cada decisión a partir de lo que recuerda del TP5. El registro está en `docs/prompts.md`.